Design and Implementation of a Secure University Information Management Portal with Role-Based Access Control Using Python

Final Year Project Report — Department of Computer Science, Faculty of Natural and Applied Sciences, Legacy University, Okija, Anambra State, Nigeria.

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Legacy University, Okija, sits on the Onitsha-Owerri Expressway in Anambra State and has been running since 2016. Like a good number of relatively young Nigerian private universities, most of its day-to-day administrative work still happens the way it would have happened twenty years ago: registration forms filled out by hand or emailed as spreadsheets, results collated in departmental offices before they ever reach a student, and departmental records kept wherever the person in charge of them happens to keep them. The university's public website gives prospective students information about the institution, but it stops there. There is no student portal, no staff portal, nothing that lets a registered student log in and see their own course list or a lecturer submit a result without physically handing over a sheet of paper.

This is not unusual for a young institution still building out its administrative infrastructure, but it creates a specific kind of friction that shows up most sharply during registration periods and whenever results are due. A registrar's office working from spreadsheets has no easy way to guarantee that only the right person edited a given student's record, or to prove after the fact who changed what. A student who wants to check whether their registration went through has to ask someone, in person or by phone, rather than simply looking. None of this is catastrophic on its own, but it adds up to a system that is slower than it needs to be and harder to audit than it should be.

Role-Based Access Control (RBAC) is a well-established answer to the access-control half of that problem. Rather than everyone with a login being able to see or touch everything, RBAC ties what a user can do to the role they hold, so a student's account is mechanically incapable of editing another student's results no matter what URL they type into the browser. Ferraiolo and Kuhn's early formalization of the model, and the NIST RBAC standard that grew out of Sandhu et al.'s later work, are still the reference point most practical systems build from, and Django, the framework this project is built on, already ships with a group-based permission system that maps onto RBAC reasonably well.

This project takes that model and applies it to Legacy University's specific situation: a working web portal, built with Python and Django, that gives each class of user (students, lecturers, heads of department, the registrar, and so on) a dashboard scoped to what they actually need to do, backed by a database that enforces those boundaries on the server rather than trusting the browser to hide the parts a user shouldn't see.

1.2 Problem Statement of the Study

Legacy University currently has no functional system for managing academic records, course registration, or results processing. Everything runs on paper, spreadsheets, or informal digital tools with no consistent rule governing who is allowed to view or change a given piece of institutional data. Three problems fall directly out of that.

First, there is no reliable access boundary. Anyone with access to a shared spreadsheet can, in principle, see or edit records that have nothing to do with them, and there is no mechanism forcing a lecturer's changes to stay within their own courses or a student's view to stay within their own record.

Second, the processes themselves are slow. Course registration and result checking both depend on someone being available to process a form or answer a query, which means students wait, and staff spend time on repetitive administrative work that a system could handle on its own.

Third, and this follows from the first two, there is effectively no record of who did what. If a result is entered incorrectly or a registration goes missing, there is rarely a clean way to trace how it happened, since the underlying tools were never built with that kind of accountability in mind.

1.3 Aim and Objective of the Study

Aim: to design and implement a secure, web-based University Information Management Portal for Legacy University, built around role-based access control, that replaces the manual processes described above for the modules within its scope.

Objectives:

1. To design a role-based access control model covering the university's key user roles (Student, Lecturer, Head of Department, Registrar, Bursar, Dean, and IT Administrator) and enforce it at the application layer.
2. To implement a secure authentication system, including a passwordless first-login flow for students (verified by an emailed PIN) and an emailed setup link for staff, so no account ever starts life with a shared default password.
3. To build a student information module covering profile management, self-service account changes, and department/faculty administration.
4. To build a course registration module that lets students register for courses scoped to their department and level, with validation against the department's programme structure.
5. To build a results and grading module that lets lecturers enter scores against their assigned courses, with automatic grading and GPA/CGPA computation on the NUC's five-point scale.
6. To evaluate the resulting system against the requirements set out in the accompanying Software Requirements Specification, and to document where the final build differs from the original design.

1.4 Significance of the Study

For students, the portal removes the need to physically track down staff to register for a course or find out whether a result has been published; both become something they can check for themselves, at any time, from their own account.

For the registrar's office and departmental staff, the system centralizes records that currently live across spreadsheets and paper files, and it removes a category of error that comes from copying the same information between different manual records.

For the Department of Computer Science, the project is a working demonstration that a role-based access control system, correctly scoped, is something a single student developer can design and build within a final year project's timeline. That has some bearing on whether a wider, institution-backed version of the idea is worth pursuing later, which is the question the accompanying pilot proposal puts to the department directly.

For the researcher, the project is a chance to work through the practical gap between how RBAC is described in the literature and what it actually takes to enforce it in a real, running application. Some of that gap turned out to be smaller than expected, and some of it, documented honestly in Chapter Three and Chapter Four, turned out to be larger.

1.5 Scope of the Study

The system covers four modules, in line with the scope agreed with the department before implementation began:

- **Authentication and role-based access control** — login, session management, account lockout, and the per-role permission checks that govern every other module.
- **Student information** — student profiles, department and faculty records, and self-service account management (preferred username, password recovery, email changes).
- **Course registration** — course listings scoped to department and level, and semester registration by students.
- **Results and grading** — score entry by lecturers, automatic grading on the NUC five-point scale, and GPA/CGPA computation for students.

Seven roles are modelled in the RBAC layer: Student, Lecturer, Head of Department, Registrar, Bursar, Dean, and IT Administrator. Not every role has a fully built-out feature set behind it; the Bursar role, for instance, exists as a real, working login role with its own dashboard, but no fee-management functionality sits behind it in the current build. Fee management, transcript processing, attendance tracking, and system-wide audit logging were all part of the original seven-role, nine-module design discussed in Chapter Three, but none of them made it into the final implementation. Where that matters for how a chapter should be read, it is flagged explicitly rather than left for the reader to discover on their own.

1.6 Limitations of the Study

The system was built and tested by a single developer within the time available for a final year project, which set a hard ceiling on how much of the originally-planned scope could realistically be delivered; Section 1.5 above and Chapter Three's discussion of scope evolution both go into what was cut and why.

The database in use is SQLite, chosen for its zero-configuration convenience during development. No PostgreSQL configuration was ever added, so the "production-ready" database swap discussed in some of the project's earlier planning documents did not happen in practice.

The system has not been deployed to a live production environment or trialled with real institutional data; it was built and tested against realistic but synthetic student, course, and result records. The accompanying pilot proposal to the department outlines what a real, limited-scope trial would look like, but that trial had not started as of this report.

No payment gateway is integrated, since fee management was outside the final scope. The system is browser-based only, with no native mobile application.

1.7 Definition of Terms

RBAC (Role-Based Access Control): a security model in which a user's ability to view or act on data is determined by the role or roles assigned to their account, rather than by permissions attached to the individual user.

MVT (Model-View-Template): the architectural pattern Django applications follow — a Model layer for data, a View layer for request handling and business logic, and a Template layer for rendering HTML.

CRUD: shorthand for the four basic data operations a system supports — Create, Read, Update, Delete.

HOD: Head of Department, one of the seven roles modelled in the system.

CSRF (Cross-Site Request Forgery): an attack that tricks a logged-in user's browser into submitting a request they didn't intend to make; Django's CSRF middleware protects every form in the portal against it.

FR: Functional Requirement, as used throughout the Software Requirements Specification (Appendix, or see docs/SRS_Legacy_University_Portal.md) and referenced by ID (e.g. FR-AUTH-05) where a design decision ties back to a specific requirement.

NUC: the National Universities Commission, the Nigerian regulatory body whose five-point grading scale (A through F, with grade points 5 down to 0) the results module implements.

GPA / CGPA: Grade Point Average and Cumulative Grade Point Average, the semester and running-total measures of a student's academic performance computed from their graded results.

CHAPTER TWO

LITERATURE REVIEW

2.1 Theoretical Review

Access control as a formal subject predates RBAC by decades, and it's worth starting there rather than jumping straight to the model this project uses. Saltzer and Schroeder's 1975 paper, *The Protection of Information in Computer Systems*, is still the reference point most later work builds on. It set out a small number of design principles for protection mechanisms: least privilege, complete mediation, fail-safe defaults, economy of mechanism, separation of privilege, and a few others less directly relevant here. Least privilege in particular, the idea that a user or process should hold no more access than the task in front of it requires, is close to the founding intuition behind RBAC, even though Saltzer and Schroeder weren't describing role-based systems specifically. The decorator-based permission checks in this project's `accounts` app, each denying access unless a request comes from a user holding the specific role a view requires, are a fairly direct application of that principle at the code level.

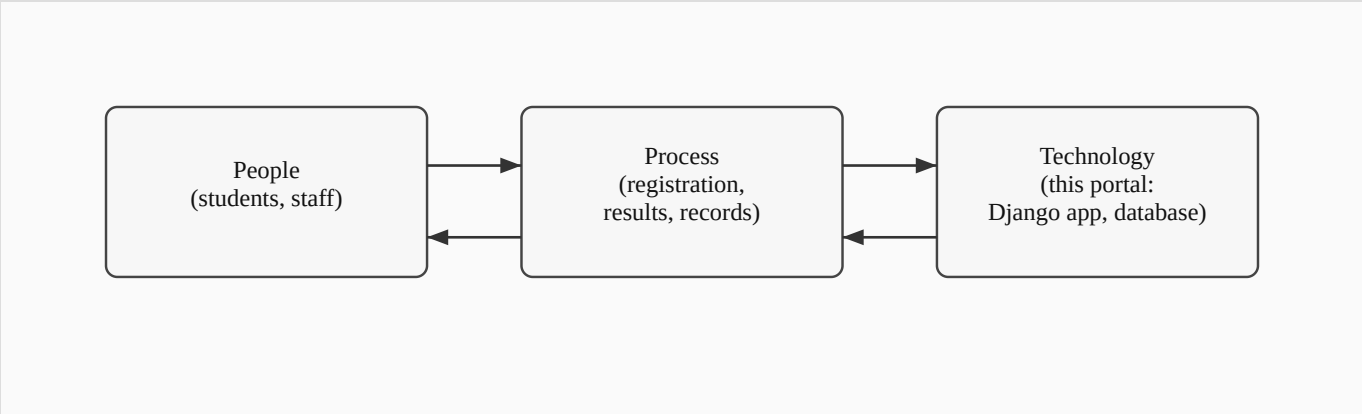
Role-Based Access Control itself was formalized by Ferraiolo and Kuhn in their 1992 paper for the National Computer Security Conference. Their argument was that neither of the two dominant models at the time, discretionary access control (where the resource owner decides who gets access) and mandatory access control (where a central authority assigns fixed security labels), matched how organizations actually managed access in practice. Most organizations think in terms of job functions: a registrar does registrar things, a lecturer does lecturer things, and access should follow that structure rather than being assigned person by person or locked to a rigid label. RBAC introduces the role as an intermediate layer between users and permissions, so permissions attach to roles and users are assigned to roles, rather than permissions attaching to individual users directly.

Sandhu, Coyne, Feinstein, and Youman took that idea further in 1996, publishing a family of four reference models in IEEE Computer, usually referred to as RBAC96. The simplest, flat RBAC, is close to what this project implements: users hold roles, roles hold permissions, and that's the whole picture. Their more elaborate models add role hierarchies (where a senior role inherits a junior role's permissions) and constraints (such as mutually exclusive roles, where the same user can never hold two conflicting roles at once). This project doesn't need role hierarchies or exclusion constraints. Its seven roles are largely independent of one another, and a Django Group per role, checked by one decorator per view, covers the requirement without the added complexity RBAC96's fuller models bring.

Beyond access control specifically, Laudon and Laudon's *Management Information Systems: Managing the Digital Firm* frames the broader point of a system like this one: an information system isn't just software, it's a combination of people, process, and technology built to solve an organizational problem.

That framing matters for how Chapter Three approaches the existing (manual) system and the proposed one. The technology is the part this report spends the most time on, but the actual problem being solved, slow and error-prone administrative processes at a specific university, is organizational first and technical second.

Figure 2.1: Basic System Components *(sketch below — to be redrawn for final submission)*



Every basic information system, this one included, is this same triangle: people with a job to do, a process that job follows, and technology that supports the process. RBAC sits inside the "technology" corner, but it only matters because of what it does for the other two: it changes how the process runs (an unauthorized edit is refused rather than merely discouraged) and what people can rely on (a student trusts their result once entered, without needing to double-check it against a paper copy somewhere else).

Stallings and Brown's *Computer Security: Principles and Practice* rounds out the theoretical base, covering the practical side of authentication, session management, and the general shape of access-control enforcement in a running system, rather than the RBAC model in isolation. Several of the concrete choices in Chapter Four, session expiry after a period of inactivity, an account lockout after repeated failed logins, CSRF protection on every form, trace back to standard practice as described in that text rather than to any one paper.

2.2 Review of Related Literature

Within a Nigerian tertiary-education setting specifically, RBAC has already been applied, though narrowly. Onashoga, Abayomi-Alli, and Ogunseye (2014), working out of the Federal University of Agriculture, Abeokuta, built a Role-Based Examination System aimed at a fairly specific problem: examination questions and results being exposed to users who had no business seeing them, because the systems handling those documents had no real access control layered on top of basic authentication. Their system combined two authentication techniques with role-based checks to keep exam materials scoped to the people who should see them at each stage of the process, and it's a useful data point that RBAC as a concept has already been tested, successfully, in a Nigerian university context. What it doesn't do is extend

that model past examinations into the wider set of administrative functions, course registration, student records, results publication, that a full institutional portal has to cover. That's the gap this project sits in.

Looking at what's already running at other Nigerian universities gives a sense of what a fuller system looks like once resources aren't a constraint. The University of Nigeria, Nsukka runs a student portal (unnportal.unn.edu.ng) covering course registration, fee payment, result checking, admission status, transcript requests, the academic calendar, and admission deferment, among other functions. Covenant University's portal (portal.covenantuniversity.edu.ng) covers much the same ground, plus hostel allocation and online clearance. Both are broad, mature systems built for large student populations at well-established universities, and neither publishes any account of the access-control architecture underneath the features a student or staff member actually sees. That's a reasonable choice for a production system not built to be studied, but it also means there's no public record of how either institution actually structured its role-based permissions, if RBAC is even the model either one uses.

Legacy University sits in a different position from both of the reference institutions above: newly established, smaller, and without the budget or in-house development team a UNN or a Covenant University can draw on. A portal built for it needs to solve the same class of problem, centralizing records, scoping access by role, cutting down on manual paperwork, at a scale and cost that a single developer working within a final year project timeline can actually deliver. None of the case studies reviewed here address that specific combination: a smaller Nigerian institution, a documented RBAC design rather than an opaque production system, and a scope deliberately kept small enough to be buildable and verifiable within an academic project's constraints.

2.3 Summary of Literature Review (in Tabular Form) and Knowledge Gap

Author(s) / Year	**Focus**	**Relevance to this project**
Saltzer & Schroeder (1975)	Design principles for protection mechanisms in computer systems	Grounds the least-privilege reasoning behind the per-role decorator design in Chapter Four
Ferraiolo & Kuhn (1992)	Formalizes Role-Based Access Control as an alternative to DAC/MAC	Source of the core model this project's RBAC layer implements
Sandhu, Coyne, Feinstein & Youman (1996)	RBAC96 family of reference models (flat, hierarchical, constrained, symmetric)	Establishes that flat RBAC, the simplest model in the family, is a legitimate and sufficient choice for a system with independent, non-hierarchical roles
Laudon & Laudon (2020)	Information systems as combinations of people, process, and technology	Frames the portal as a response to an organizational problem, not just a technical exercise
Stallings & Brown (2018)	Applied computer security: authentication, session	Basis for the session-timeout, lockout, and CSRF design choices covered in Chapter Four

Author(s) / Year	Focus	Relevance to this project
	management, access control in practice	
Onashoga, Abayomi-Alli & Ogunseye (2014)	RBAC applied to a Nigerian university's electronic examination system	Confirms RBAC's feasibility in a Nigerian tertiary context, but scoped narrowly to examinations
UNN and Covenant University portals (case studies)	Large-scale, feature-rich institutional student portals	Show what a fully-resourced portal covers, but neither documents its underlying access-control design

Taken together, the literature establishes RBAC as a sound, well-studied model, shows it already working in at least one Nigerian academic context, and shows what a large-scale institutional portal looks like once resources aren't a limiting factor. What it doesn't offer is an example that sits in the middle: a documented, openly-designed RBAC system built at the scale a single developer can realistically deliver, for an institution the size of Legacy University rather than an already-established, well-resourced one. That's the specific gap this project addresses, and Chapter Three picks up from here with an analysis of Legacy University's existing (manual) system against the one proposed to replace it.

CHAPTER THREE

METHODOLOGY AND SYSTEM ANALYSIS

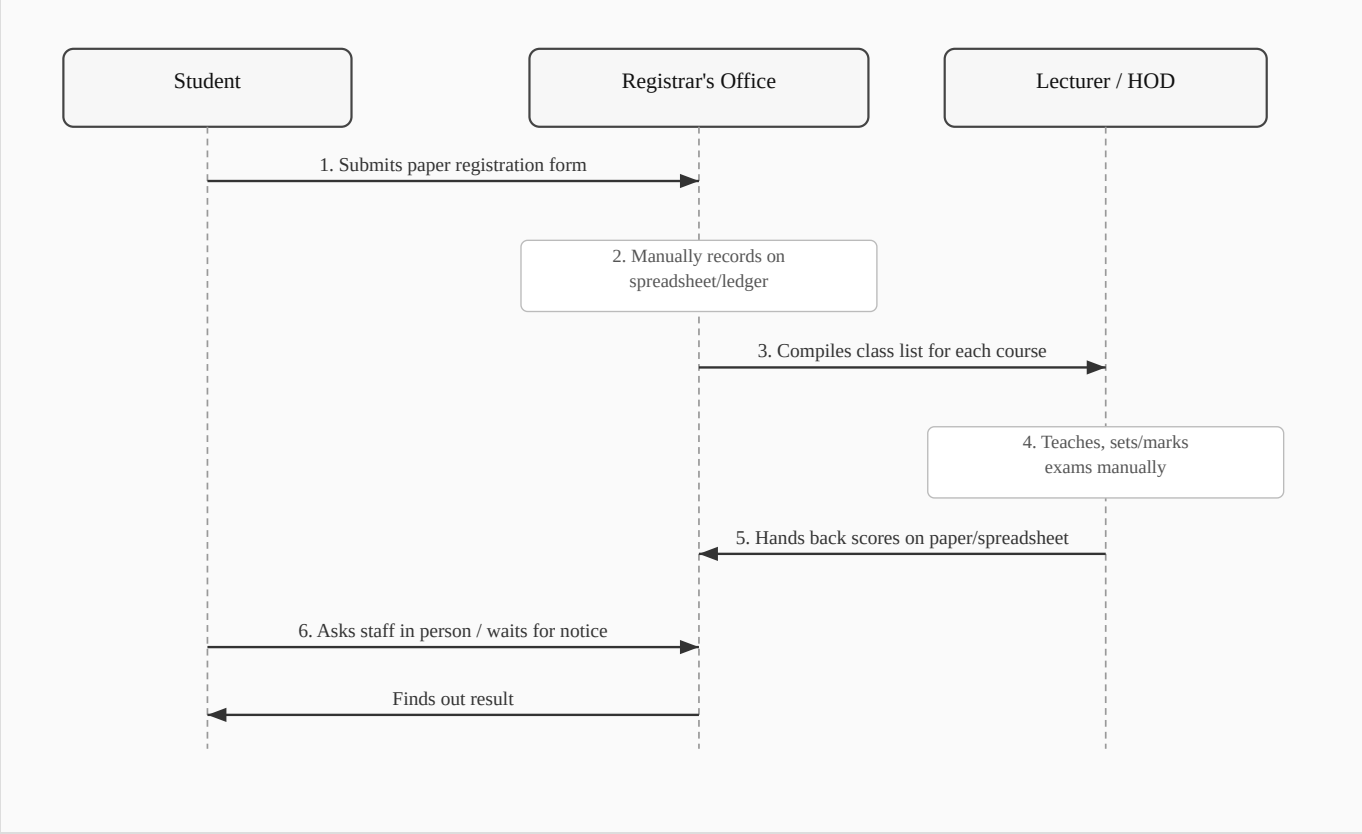
3.1 System Analysis

System analysis, here, means two things done in sequence: first understanding how Legacy University currently handles the processes this project touches, well enough to say precisely what's wrong with it, and second working out what a replacement needs to do to actually fix those problems rather than just moving them onto a screen. The two sections below cover each in turn.

3.1.2 Analysis of the Existing System

Legacy University's current process for course registration, results, and student record-keeping is entirely manual, built around paper forms, spreadsheets kept by individual staff members, and in-person coordination between students, lecturers, and the registrar's office. There is no single system of record; the closest thing to one is whatever spreadsheet a given staff member last updated.

Figure 3.1: Dataflow of the Existing (Manual) System *(sketch below — to be redrawn as a formal DFD for final submission)*



The flow above has no built-in access control. Anyone with access to a given spreadsheet can, in principle, open and edit it regardless of whether they have any legitimate role in that particular course or department, and there is no mechanism forcing a change to stay within the boundary of who made it or why.

3.1.3 Weaknesses of the Existing System

The manual system's problems fall into a few clear categories. There's no access boundary: nothing stops a staff member from opening a record that has nothing to do with their role, and nothing logs it if they do. There's no single source of truth: the same piece of information, a student's registered courses, for instance, can exist in slightly different states across two or three different spreadsheets, and reconciling them after the fact is tedious and error-prone. Turnaround is slow, since almost every step depends on a specific person being available to process a form or answer a question in person. And there's no audit trail at all: if a result is entered wrong, or a registration disappears, there's no record showing when it happened, who touched it, or how to reverse it. None of these are unusual problems for a manual system to have; they're exactly the class of problem RBAC and a centralized database are meant to solve, which is the case this project is built to make.

3.2.1 Methodology Adopted

This project follows an **object-oriented, iterative and incremental** development approach, built and tested one working slice at a time rather than designed exhaustively up front and implemented in a single pass. That choice was less a stylistic preference than a practical necessity: the original design, discussed in Chapter One and revisited in Section 3.2.4 below, specified a broader scope than a single developer could deliver on a final year project's timeline, and it only became clear which parts of that scope were realistic to keep once the earlier increments were built, tested, and evaluated against what remained of the schedule.

Concretely, that meant building authentication and the role decorators first, since every other module depends on them; then student and department records; then course registration; then results and grading; and finally a dedicated security-hardening pass once the functional modules were in place. Each increment came with its own automated test coverage, using Django's `TestCase` framework, rather than being verified by hand and left untested, so that later changes could be checked against earlier behaviour rather than re-verified manually every time. Git version control tracked each increment as its own set of commits, which is also how the scope changes documented in Section 3.2.4 and in the accompanying SRS/SDD can be traced concretely rather than just asserted.

The object-oriented half of that description reflects Django's own architecture more than a deliberate methodological choice: every entity in the system, a user, a course, a result, is modelled as a class with its own fields and behaviour, and the relationships between them (a student has one department, a course

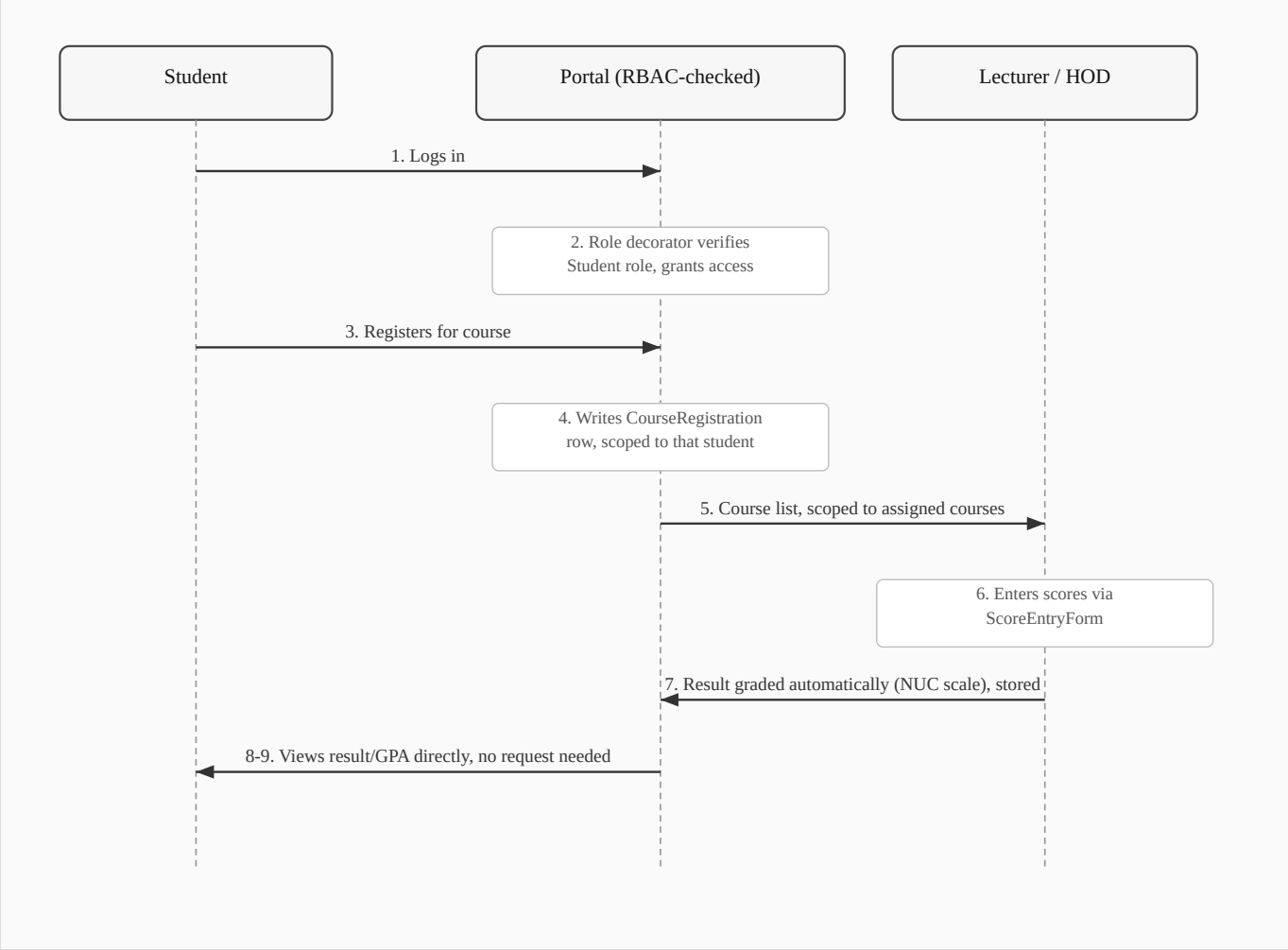
belongs to one department, a result belongs to one registration) are expressed as object relationships enforced by the ORM rather than as loose foreign keys managed by hand.

3.2.2 Analysis of the Proposed System

The proposed system replaces the manual flow in Section 3.1.2 with a single Django application backed by a role-based access control layer. Four modules make up the system as built: authentication and RBAC, student information, course registration, and results/grading, covering seven roles (Student, Lecturer, HOD, Registrar, Bursar, Dean, IT Admin) as detailed in the SRS.

Every request that touches protected data passes through a role check before any view logic runs, which closes the access-boundary gap identified in Section 3.1.3 directly: a Lecturer's account is mechanically incapable of editing a course they aren't assigned to, not because of a convention staff are expected to follow, but because the decorator guarding that view checks for it on every request. Student records, course data, and results all live in one database rather than scattered spreadsheets, so there's exactly one place a given fact about a student or a course can be found, and results are graded automatically against the NUC's five-point scale the moment a lecturer enters a score, removing a step that used to depend on someone doing the arithmetic by hand.

Figure 3.2: Dataflow Diagram of the Web-Based (Proposed) System *(sketch below — to be redrawn as a formal DFD for final submission)*



Compared with Figure 3.1, the same underlying steps happen (register, teach, grade, publish), but every arrow in this version passes through a role check before it's allowed to touch the database, and step 8, checking a result, needs no human intermediary at all.

3.2.3 Overall Use Case Diagram of the New System

Figure 3.3: Overall Use Case Diagram of the New System *(the formal UML diagram is drawn separately for final submission; the actor/use-case breakdown below is its content in tabular form, and matches Section 7 of the SRS)*

Actor	Primary use cases
Student	Log in (password or first-login PIN); view dashboard; register for courses; view results and GPA/CGPA; manage own account (preferred username, password, email)
Lecturer	Log in; view assigned courses; enter and update results
HOD	Log in; manage departmental courses
Registrar	Log in; register students individually or via bulk import; edit student profiles; search student records
Dean	Log in; view courses across faculty departments (read-only)

Actor	Primary use cases
Bursar	Log in; view dashboard (no further functionality implemented — see Section 3.2.4 and Chapter One, Section 1.5)
IT Admin	Log in; manage staff accounts; manage faculties and departments

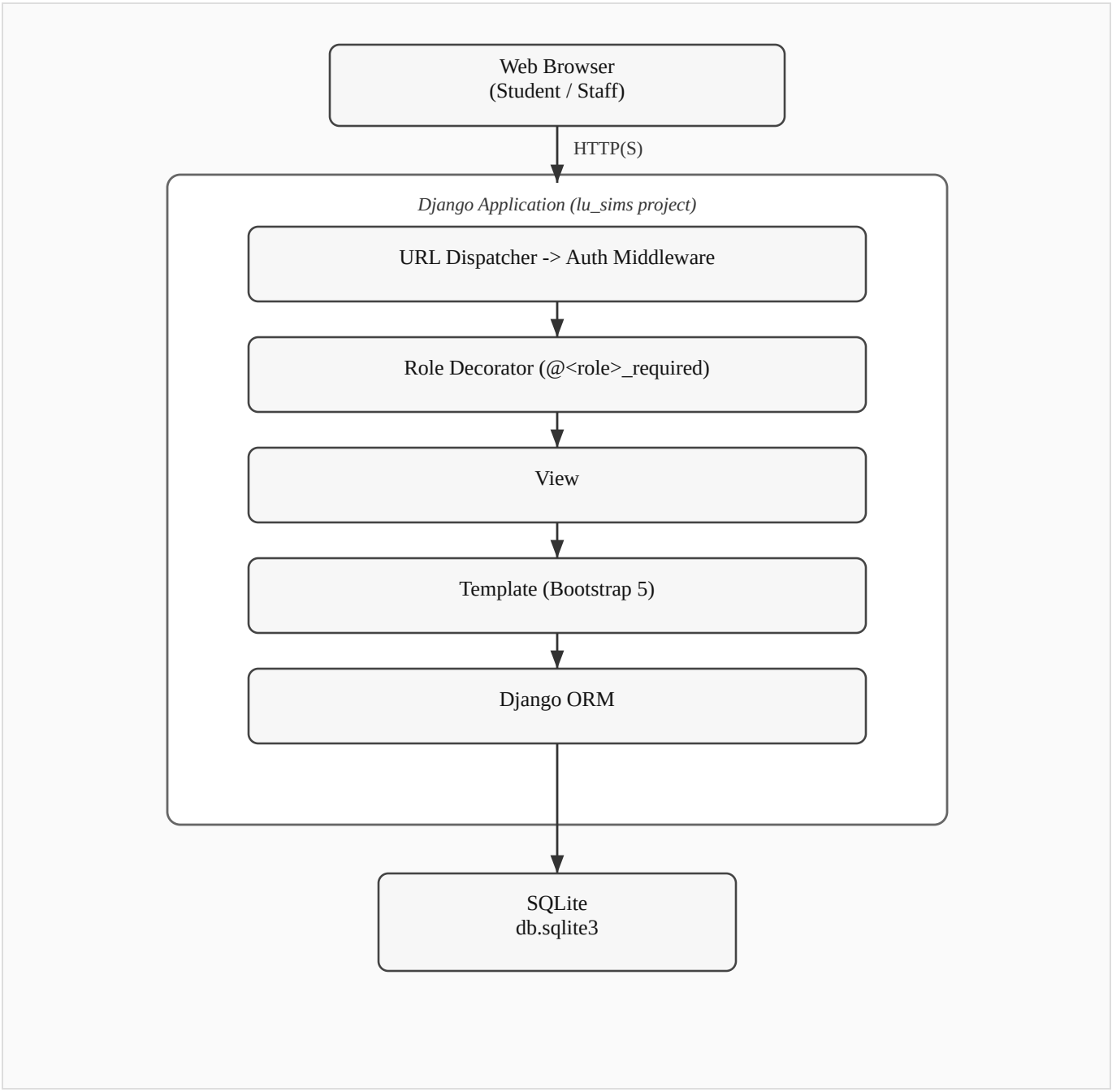
3.2.4 Justification of the New System

The case for replacing the manual process isn't just that a computer is faster than a filing cabinet; it's that RBAC, specifically, closes the access-control gap that a plain digitized spreadsheet wouldn't. A spreadsheet uploaded to a shared drive is still a spreadsheet: anyone with the link can open it, and nothing about the format stops them from editing a field that isn't theirs to touch. Building the same data into a system where every view is guarded by a role check moves that boundary from a social convention ("please don't edit records outside your department") to something the application itself enforces on every request.

The narrower scope actually delivered, four modules instead of the nine originally planned, is a direct consequence of the iterative methodology in Section 3.2.1: rather than attempting the full seven-role, twenty-one-permission design and risking an unfinished system in every module, effort concentrated on a smaller set of modules built to the point of being genuinely usable. Section 1.5 and Section 5.2 of the SRS document exactly what was cut and why. The Results/Grading module in particular replaced the originally-planned Attendance module, since it demonstrates the RBAC model across three roles at once (Student viewing, Lecturer entering, HOD's department scoping the course) rather than the single-role read/write pattern attendance tracking would have involved, making it the stronger choice for a project meant to showcase role-based access control specifically.

3.3.3 High Level Model of the New System

Figure 4.1: High Level Model of the New System *(numbered per the department's List of Figures, which sequences this diagram under Chapter Four even though the section itself is 3.3.3; sketch below — to be redrawn as a formal architecture diagram for final submission)*



A request reaches the URL dispatcher first, then Django's own authentication middleware, which redirects anyone unauthenticated straight to the login page. Past that point, the role decorator on the matching view checks the current user's role before any business logic executes; if the check fails, the request is denied with a 403 rather than being allowed to fall through to the view. Only once both checks pass does the view run, read or write through the ORM, and render a template back to the browser. This is the same request flow described in more technical detail in Chapter Four and in Section 2 of the accompanying SDD; this figure gives the same picture at the level System Analysis needs, without the implementation specifics that belong in the design chapter.